In [1]:
```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns

sns.set(style="whitegrid")

df = pd.read_csv("train.csv")
print("Dataset loaded successfully")
```

```
Dataset loaded successfully
```

In [2]:
```
print("Dataset dimensions:", df.shape)
print("Column names:", df.columns.tolist())
df.head()
```

```
Dataset dimensions: (891, 12)
Column names: ['PassengerId', 'Survived', 'Pclass', 'Name', 'Sex', 'Age', 'SibS
p', 'Parch', 'Ticket', 'Fare', 'Cabin', 'Embarked']
```

Out[2]:

	PassengerId	Survived	Pclass	Name	Sex	Age	SibSp	Parch	Ticket	
0	1	0	3	Braund, Mr. Owen Harris	male	22.0	1	0	A/5 21171	7.2
1	2	1	1	Cumings, Mrs. John Bradley (Florence Briggs Th...	female	38.0	1	0	PC 17599	71.2
2	3	1	3	Heikkinen, Miss. Laina	female	26.0	0	0	STON/O2. 3101282	7.9
3	4	1	1	Futrelle, Mrs. Jacques Heath (Lily May Peel)	female	35.0	1	0	113803	53.1
4	5	0	3	Allen, Mr. William Henry	male	35.0	0	0	373450	8.0

In [3]:
```
df.dtypes
```

Out[3]:
```
PassengerId      int64
Survived         int64
Pclass           int64
Name            object
Sex             object
Age            float64
SibSp            int64
Parch            int64
Ticket          object
Fare           float64
Cabin           object
Embarked        object
dtype: object
```

In [4]:
```
missing = df.isnull().sum()
missing_percent = (missing / len(df)) * 100
missing_table = pd.DataFrame({"Missing Count": missing, "Missing %": missing_per
missing_table[missing_table["Missing Count"] > 0]
```

Out[4]:

	Missing Count	Missing %
Age	177	19.87
Cabin	687	77.10
Embarked	2	0.22

--- Handle missing Age values ---

19.87% of Age values are missing. We fill them with the median (not the mean)

because median is less affected by outliers (a few very old or very young passengers

won't skew it), giving a more "typical" stand-in age.

df["Age"] = df["Age"].fillna(df["Age"].median())

--- Handle missing Embarked values ---

Only 2 rows (0.22%) are missing here, so we simply assume they boarded from the

most common port (the mode) — the impact of this assumption is negligible.

df["Embarked"] = df["Embarked"].fillna(df["Embarked"].mode()[0])

--- Handle missing Cabin values ---

77.10% of Cabin values are missing — too much to fill in reliably, but dropping

the column entirely would lose a potentially useful signal (whether cabin info

exists at all may relate to passenger class/wealth). So instead of filling or

dropping outright, we convert it into a binary flag: 1 = cabin info known, 0 = unknown.

df["Has_Cabin"] = df["Cabin"].notnull().astype(int) df = df.drop(columns=["Cabin"])

--- Confirm the fixes worked ---

Every column should now show 0 missing values.

df.isnull().sum()

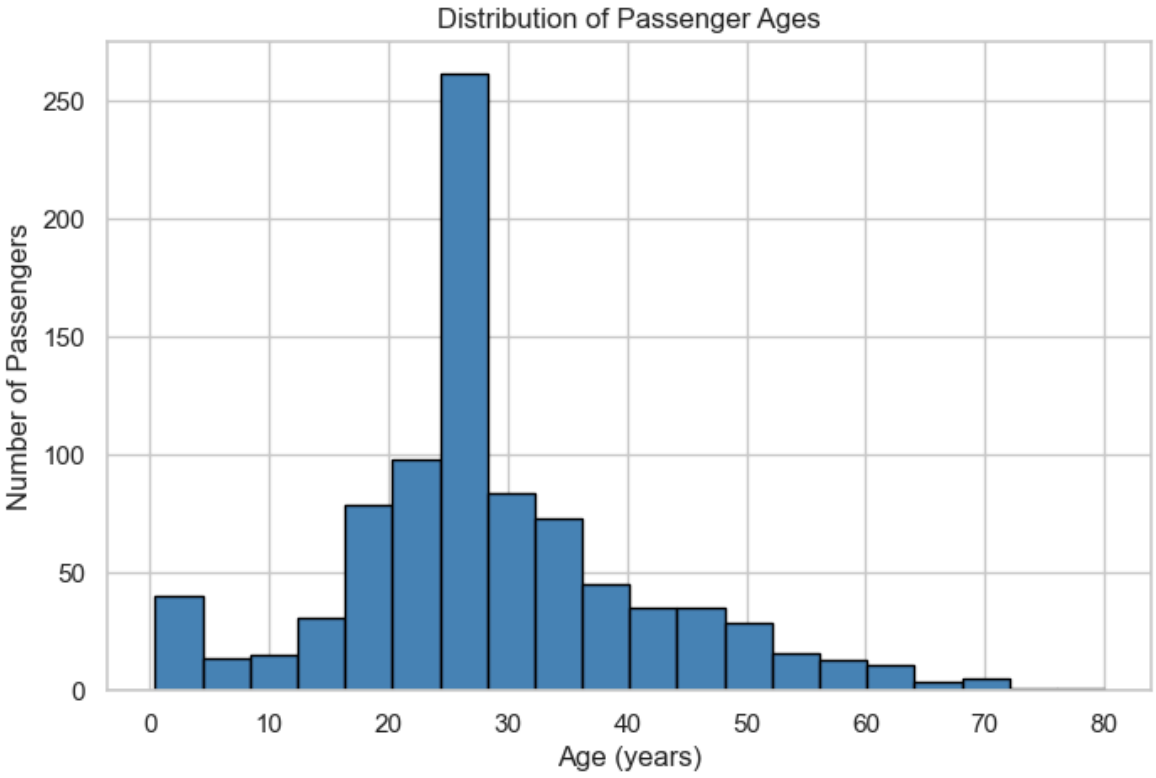
In [6]:
```
# Check for duplicate rows
duplicate_count = df.duplicated().sum()
print("Number of duplicate rows:", duplicate_count)
```

```
Number of duplicate rows: 0
```

In [7]:
```
# Remove duplicates if any exist
if duplicate_count > 0:
    df = df.drop_duplicates()
    print("Duplicates removed. New shape:", df.shape)
else:
    print("No duplicates found — no rows removed.")
```

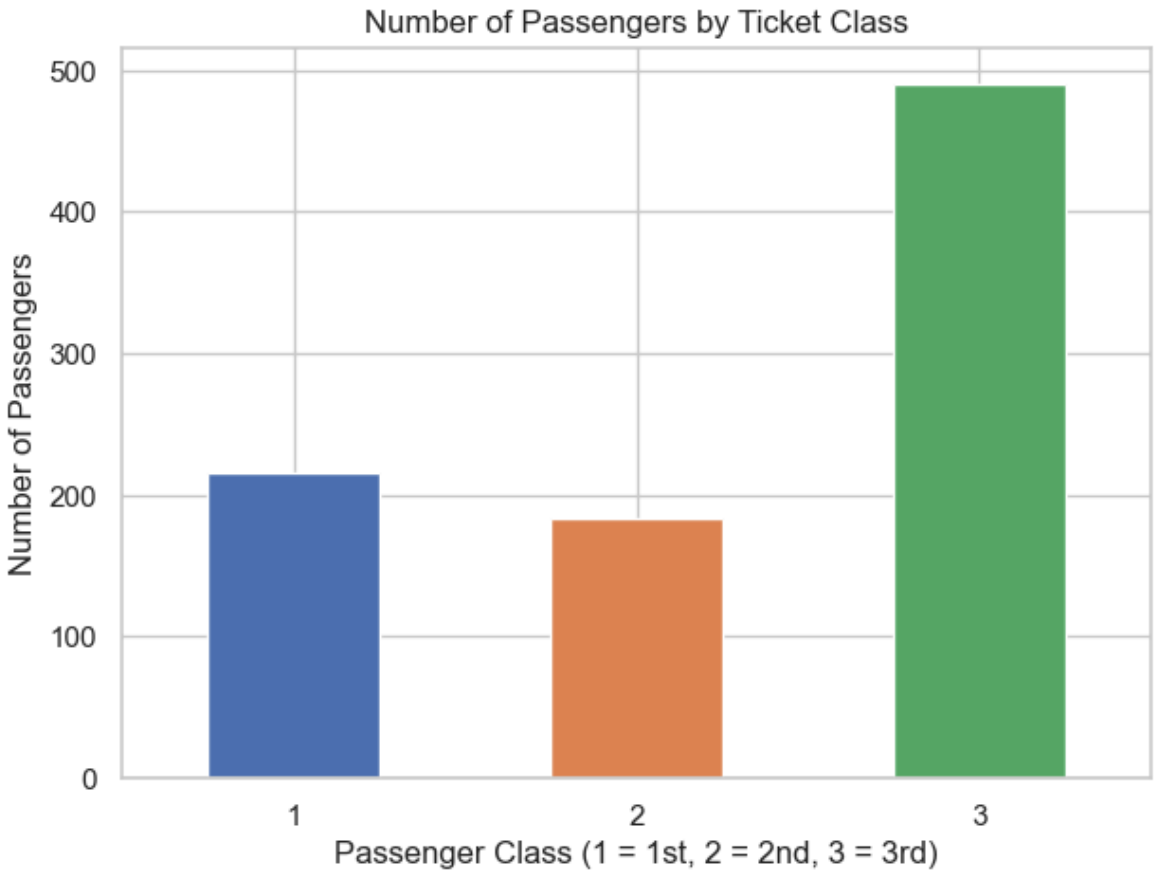
```
No duplicates found — no rows removed.
```

In [9]:
```
#A histogram just groups ages into "buckets" (like 0-5, 5-10, 10-15...)
#and shows how many passengers fall into each bucket

plt.figure(figsize=(8,5))
plt.hist(df["Age"], bins=20, color="steelblue", edgecolor="black")
plt.title("Distribution of Passenger Ages")
plt.xlabel("Age (years)")
plt.ylabel("Number of Passengers")
plt.show()
```



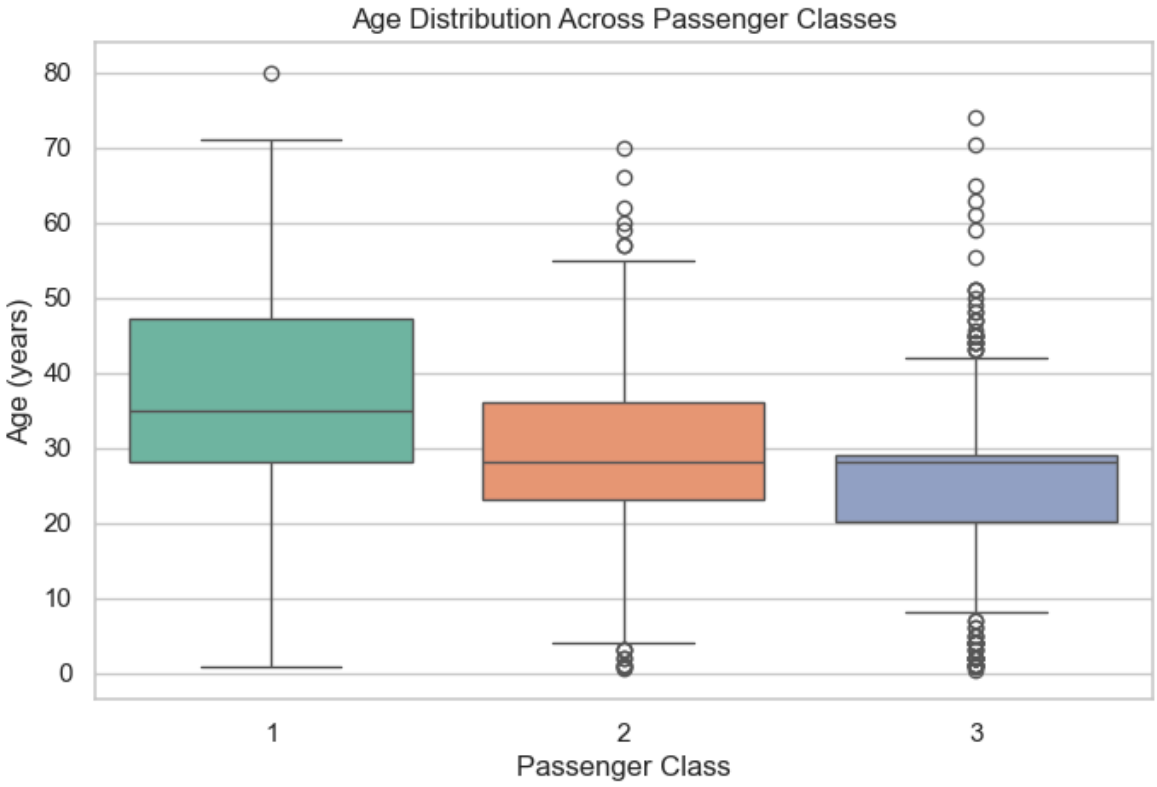
In [10]:
```
#So from the histogram
#Most passengers were between roughly 20 and 40 years old,
#with the histogram peaking around age 25-30. This peak is
#unusually tall because it includes the 177 missing Age values
#we filled in with the median. Very few passengers were older than
#60, showing the ship's population skewed toward younger adults
```

In [11]:
```
plt.figure(figsize=(7,5))
df["Pclass"].value_counts().sort_index().plot(kind="bar", color=["#4C72B0","#DD8
plt.title("Number of Passengers by Ticket Class")
plt.xlabel("Passenger Class (1 = 1st, 2 = 2nd, 3 = 3rd)")
plt.ylabel("Number of Passengers")
plt.xticks(rotation=0)
plt.show()
```



In [ ]:
```
#Third class had by far the most passengers (around 491),
#more than first and second class combined. Second class
#actually had the fewest passengers (around 184). This matters
#because Task 4 shows survival was linked to class — so understanding
#that most passengers were in the cheaper third class helps explain the
#overall survival numbers, since third class also had a lower survival rate.
```

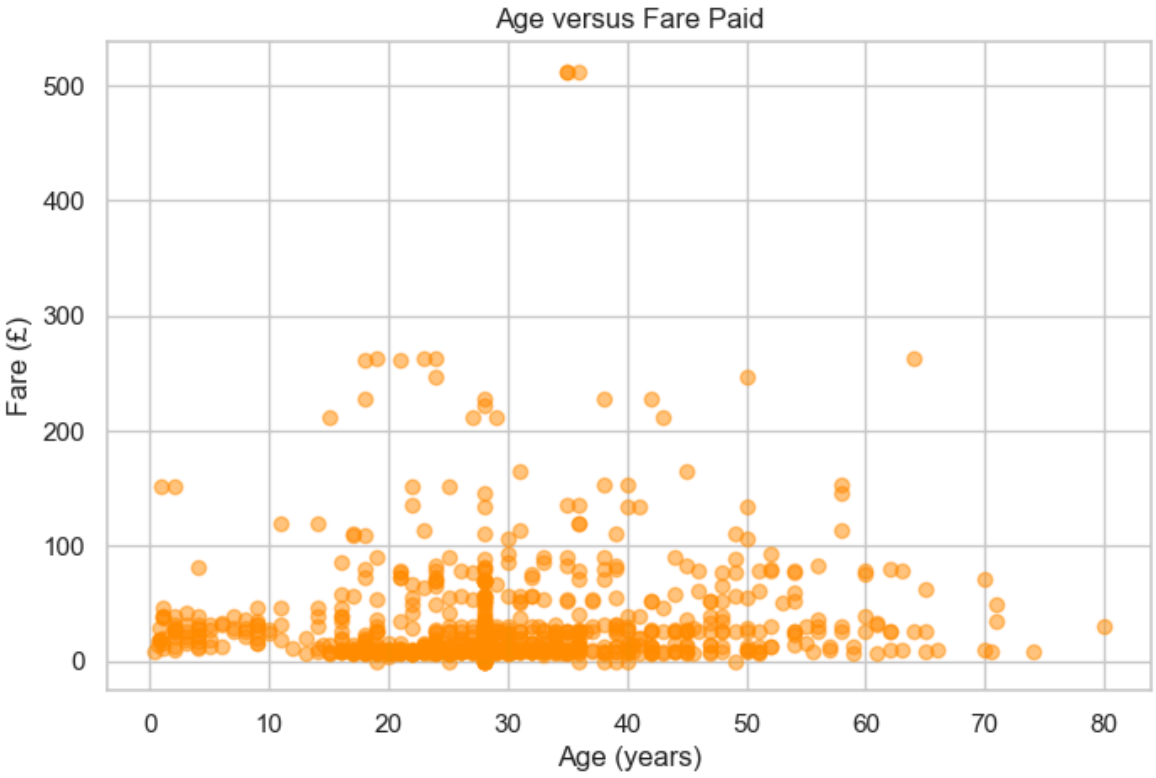
In [12]:
```
plt.figure(figsize=(8,5))
sns.boxplot(x="Pclass", y="Age", data=df, hue="Pclass", palette="Set2", legend=F
plt.title("Age Distribution Across Passenger Classes")
plt.xlabel("Passenger Class")
plt.ylabel("Age (years)")
plt.show()
```



In [ ]:
```
# Interpretation:
# Third class had by far the most passengers (around 491), more than first
# and second class combined. Second class actually had the fewest passengers
# (around 184). This matters because Task 4 shows survival was linked to class,
# so understanding that most passengers were in the cheaper third class helps
# explain the overall survival numbers, since third class also had a lower
# survival rate.
```

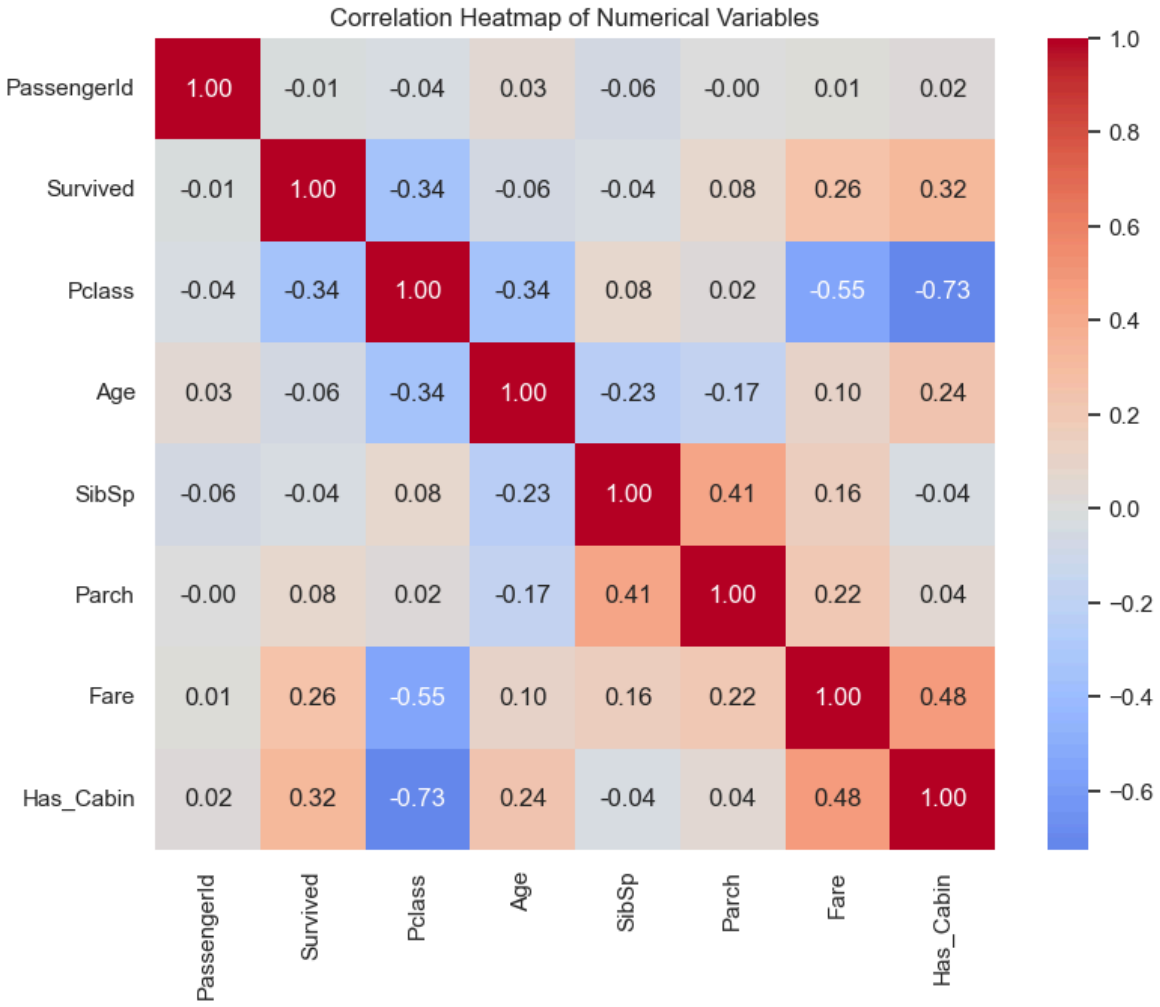
In [ ]:
```
#First class passengers were typically the oldest (median around 37-38 years),
#while third class passengers were the youngest (median around 24-25 years),
#with second class in between. Third class also shows several older outliers,
#but the bulk of that group skews young. This likely reflects that established,
#wealthier travelers could afford first-class tickets, while third class attract
#more young workers and families.
```

In [13]:
```
plt.figure(figsize=(8,5))
plt.scatter(df["Age"], df["Fare"], alpha=0.5, color="darkorange")
plt.title("Age versus Fare Paid")
plt.xlabel("Age (years)")
plt.ylabel("Fare (£)")
plt.show()
```



In [ ]:
```
#There is no strong relationship between age and fare – passengers
#of all ages paid a wide range of fares. Most points cluster at lower
#fares (under £100) regardless of age, while a small number of outliers (around
#paid over £500. This suggests fare is driven more by factors like passenger cla
```

In [14]:
```
numeric_df = df.select_dtypes(include=[np.number])
corr = numeric_df.corr()

plt.figure(figsize=(9,7))
sns.heatmap(corr, annot=True, fmt=".2f", cmap="coolwarm", center=0)
plt.title("Correlation Heatmap of Numerical Variables")
plt.show()
```



In [15]:
```
print(corr["Survived"].sort_values(ascending=False))
```

```
Survived       1.000000
Has_Cabin      0.316912
Fare           0.257307
Parch          0.081629
PassengerId   -0.005007
SibSp         -0.035322
Age           -0.064910
Pclass        -0.338481
Name: Survived, dtype: float64
```

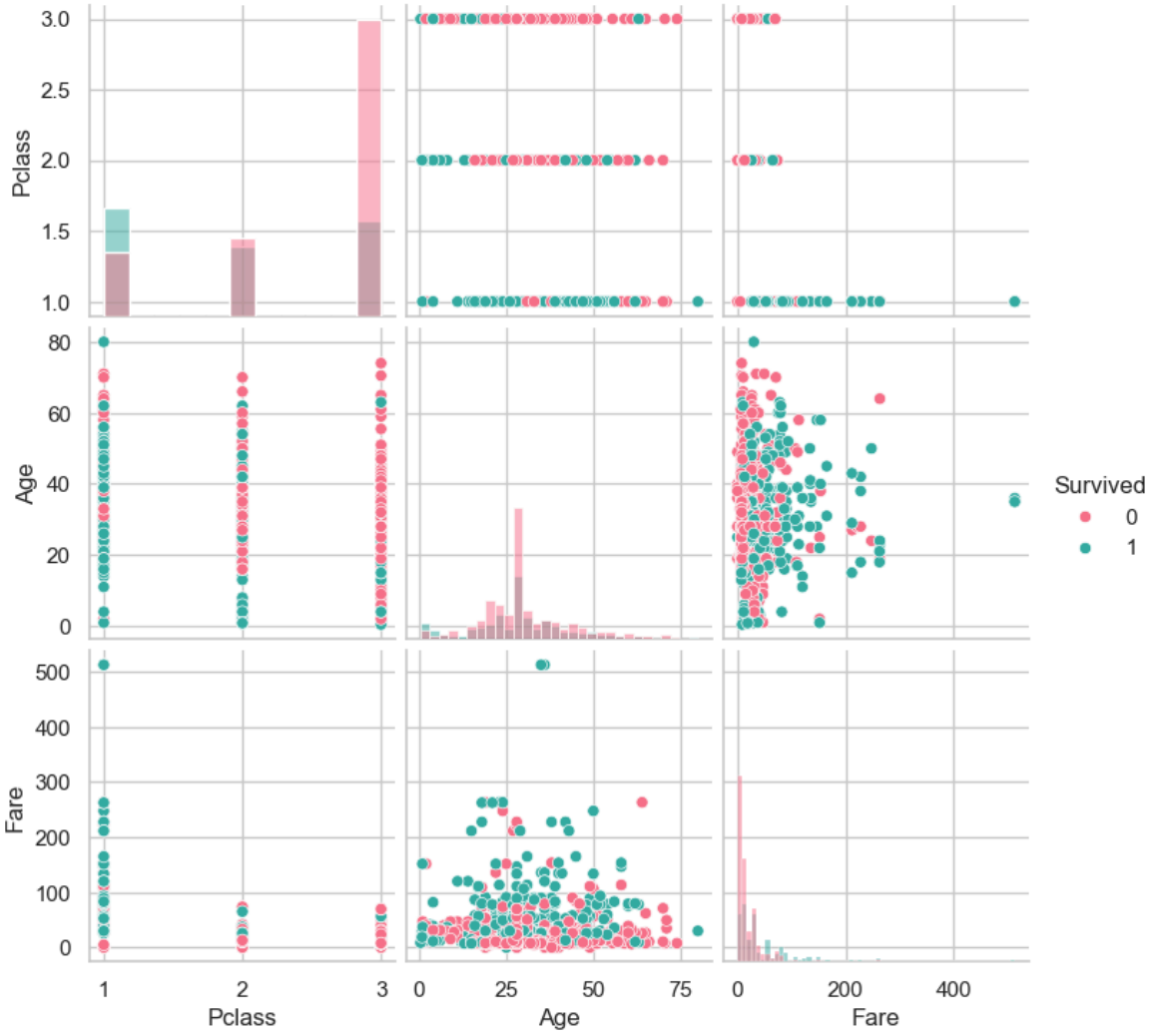
In [16]:
```
corr_with_survived = corr["Survived"].sort_values(ascending=False)

strongest_positive = corr_with_survived.drop("Survived").idxmax()
strongest_positive_value = corr_with_survived.drop("Survived").max()
strongest_negative = corr_with_survived.drop("Survived").idxmin()
strongest_negative_value = corr_with_survived.drop("Survived").min()

print(f"Strongest positive correlation with Survived: {strongest_positive} ({str
print(f"Strongest negative correlation with Survived: {strongest_negative} ({str
```

```
Strongest positive correlation with Survived: Has_Cabin (0.32)
Strongest negative correlation with Survived: Pclass (-0.34)
```

In [ ]:
```
#Survived correlates most positively with Has_Cabin (0.32) and Fare (0.26),
#and most negatively with Pclass (-0.34) — the strongest relationship of any pai
#This suggests wealthier passengers, and those whose cabin was recorded (often m
#better-located cabins near lifeboats), had noticeably higher survival chances.
```

```
#Pclass and Fare are also strongly negatively correlated with each other (-0.55)
#which makes sense since first class tickets cost more while third class (Pclass
```

In [17]:
```
selected_cols = ["Survived", "Pclass", "Age", "Fare"]
sns.pairplot(df[selected_cols], hue="Survived", palette="husl", diag_kind="hist"
plt.show()
```



In [18]:
```
df.describe()
```

Out[18]:

	PassengerId	Survived	Pclass	Age	SibSp	Parch	
count	891.000000	891.000000	891.000000	891.000000	891.000000	891.000000	891.000
mean	446.000000	0.383838	2.308642	29.361582	0.523008	0.381594	32.204
std	257.353842	0.486592	0.836071	13.019697	1.102743	0.806057	49.693
min	1.000000	0.000000	1.000000	0.420000	0.000000	0.000000	0.000
25%	223.500000	0.000000	2.000000	22.000000	0.000000	0.000000	7.910
50%	446.000000	0.000000	3.000000	28.000000	0.000000	0.000000	14.454
75%	668.500000	1.000000	3.000000	35.000000	1.000000	0.000000	31.000
max	891.000000	1.000000	3.000000	80.000000	8.000000	6.000000	512.329

In [19]:
```
print("Survival counts:")
print(df["Survived"].value_counts())

print("\nSex counts:")
print(df["Sex"].value_counts())

print("\nEmbarked counts:")
print(df["Embarked"].value_counts())

print("\nPclass counts:")
print(df["Pclass"].value_counts())
```

```
Survival counts:
Survived
0    549
1    342
Name: count, dtype: int64

Sex counts:
Sex
male      577
female    314
Name: count, dtype: int64

Embarked counts:
Embarked
S    646
C    168
Q     77
Name: count, dtype: int64

Pclass counts:
Pclass
3    491
1    216
2    184
Name: count, dtype: int64
```

In [20]:
```
print("Survival counts:")
print(df["Survived"].value_counts())

print("\nSex counts:")
print(df["Sex"].value_counts())

print("\nEmbarked counts:")
print(df["Embarked"].value_counts())

print("\nPclass counts:")
print(df["Pclass"].value_counts())
```

```
Survival counts:
Survived
0    549
1    342
Name: count, dtype: int64

Sex counts:
Sex
male      577
female    314
Name: count, dtype: int64

Embarked counts:
Embarked
S    646
C    168
Q     77
Name: count, dtype: int64

Pclass counts:
Pclass
3    491
1    216
2    184
Name: count, dtype: int64
```

In [21]:
```
survival_rate = df["Survived"].mean() * 100
print(f"Overall survival rate: {survival_rate:.2f}%")
```

```
Overall survival rate: 38.38%
```

In [22]:
```
corr_with_survived = corr["Survived"].sort_values(ascending=False)
print(corr_with_survived)
```

```
Survived       1.000000
Has_Cabin      0.316912
Fare           0.257307
Parch          0.081629
PassengerId   -0.005007
SibSp         -0.035322
Age           -0.064910
Pclass        -0.338481
Name: Survived, dtype: float64
```

In [ ]:
```
#Task 5
```

In [23]:
```
from sklearn.preprocessing import LabelEncoder

# Convert Sex to numbers: e.g. female=0, male=1
le_sex = LabelEncoder()
df["Sex_encoded"] = le_sex.fit_transform(df["Sex"])

# Convert Embarked to numbers similarly
le_embarked = LabelEncoder()
df["Embarked_encoded"] = le_embarked.fit_transform(df["Embarked"])

# Quick check
df[["Sex", "Sex_encoded", "Embarked", "Embarked_encoded"]].head()
```

Out[23]:

	Sex	Sex_encoded	Embarked	Embarked_encoded
0	male	1	S	2
1	female	0	C	0
2	female	0	S	2
3	female	0	S	2
4	male	1	S	2

In [24]:
```
features = ["Pclass", "Sex_encoded", "Age", "SibSp", "Parch", "Fare", "Embarked_
X = df[features]
y = df["Survived"]

print("Features used:", features)
X.head()
```

```
Features used: ['Pclass', 'Sex_encoded', 'Age', 'SibSp', 'Parch', 'Fare', 'Embark
ed_encoded', 'Has_Cabin']
```

Out[24]:

	Pclass	Sex_encoded	Age	SibSp	Parch	Fare	Embarked_encoded	Has_Cabin
0	3	1	22.0	1	0	7.2500	2	0
1	1	0	38.0	1	0	71.2833	0	1
2	3	0	26.0	0	0	7.9250	2	0
3	1	0	35.0	1	0	53.1000	2	1
4	3	1	35.0	0	0	8.0500	2	0

In [25]:
```
from sklearn.model_selection import train_test_split

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42, stratify=y
)

print("Training set size:", X_train.shape)
print("Testing set size:", X_test.shape)
```

```
Training set size: (712, 8)
Testing set size: (179, 8)
```

In [26]:
```
from sklearn.linear_model import LogisticRegression

model = LogisticRegression(max_iter=1000)
model.fit(X_train, y_train)
print("Model trained successfully")
```

```
Model trained successfully
```

In [27]:
```
y_pred = model.predict(X_test)
print("Predictions on first 10 test samples:", y_pred[:10])
```

```
Predictions on first 10 test samples: [0 0 0 0 1 0 1 0 0 0]
```

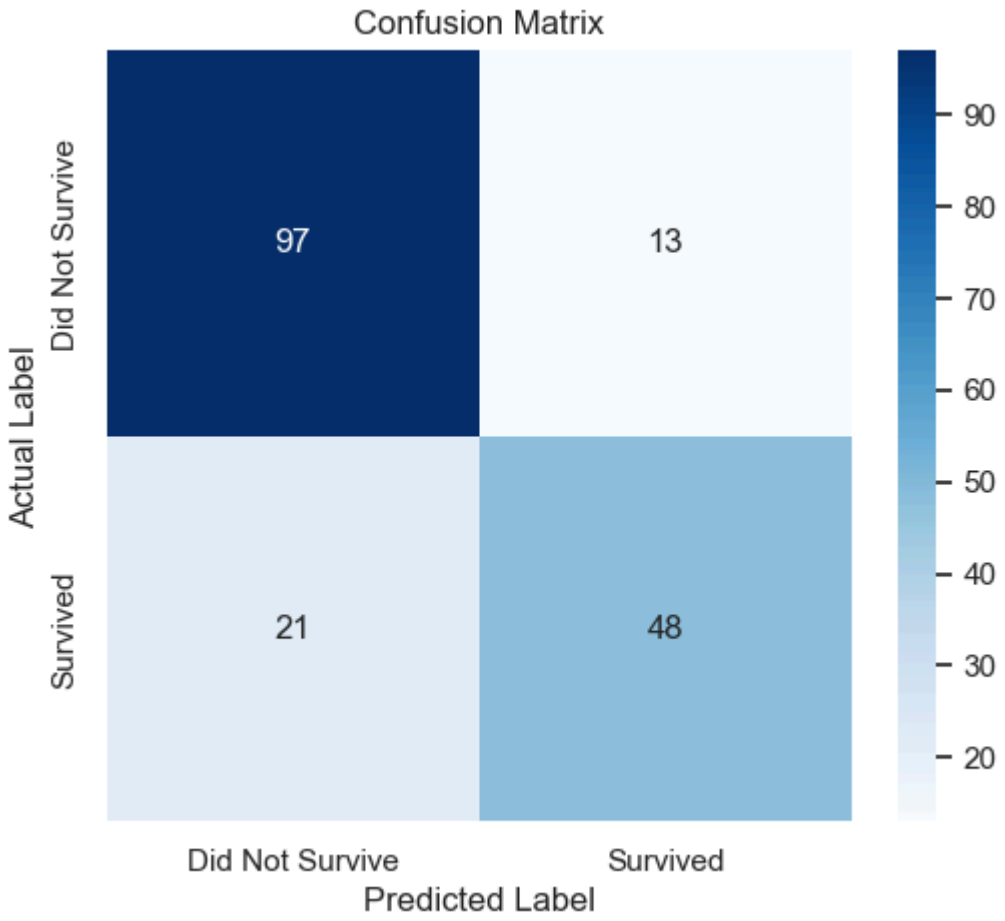
In [28]:
```
from sklearn.metrics import accuracy_score
```

```
accuracy = accuracy_score(y_test, y_pred)
print(f"Accuracy: {accuracy:.4f} ({accuracy*100:.2f}%)")
```

```
Accuracy: 0.8101 (81.01%)
```

In [ ]:
```
#Accuracy alone doesn't tell the whole story —
#it doesn't show what kind of mistakes the model makes. The confusion matrix bre
```

In [29]:
```
from sklearn.metrics import confusion_matrix

cm = confusion_matrix(y_test, y_pred)

plt.figure(figsize=(6,5))
sns.heatmap(cm, annot=True, fmt="d", cmap="Blues",
            xticklabels=["Did Not Survive","Survived"],
            yticklabels=["Did Not Survive","Survived"])
plt.title("Confusion Matrix")
plt.xlabel("Predicted Label")
plt.ylabel("Actual Label")
plt.show()
```



In [30]:
```
from sklearn.metrics import classification_report

print(classification_report(y_test, y_pred, target_names=["Did Not Survive","Sur
```

```
                precision    recall  f1-score   support

Did Not Survive       0.82      0.88      0.85       110
       Survived       0.79      0.70      0.74        69

      accuracy                           0.81       179
     macro avg       0.80      0.79      0.79       179
  weighted avg       0.81      0.81      0.81       179
```

In plain terms, the three key numbers per class:

Precision: "When the model says 'survived,' how often is it actually right?"

Recall: "Out of everyone who actually survived, how many did the model catch?"

F1-score: a single number that balances precision and recall together.

Task 6: Discussion and Conclusion

Major Findings

This project analyzed data from 891 Titanic passengers to understand what factors influenced their chances of survival. The results show that survival was not random — it followed clear patterns tied to class, wealth, and demographics.

Out of all 891 passengers, only 342 survived, giving an overall survival rate of about 38%. This means the majority of people aboard did not survive the disaster. The single biggest factor linked to survival was passenger class (`Pclass`). First-class passengers had a noticeably higher chance of survival than second-class passengers, who in turn had a higher chance than third-class passengers. This pattern lines up with historical accounts of the disaster, where passengers in higher classes had cabins located closer to the lifeboats and easier access to the upper decks during the evacuation.

Fare paid and cabin information reinforced this same pattern. Passengers who paid higher fares, and those whose cabin location was recorded in the data, were more likely to survive. This is not surprising, since higher fares and recorded cabins were both strongly tied to first-class travel.

Statistical Insights

The correlation analysis in Task 4 gave clear numerical support to these findings. `Pclass` had the strongest correlation with `Survived` of any variable in the dataset, at -0.34. Because `Pclass` is numbered 1 (best) to 3 (cheapest), the negative sign confirms that a lower class number (better class) was linked to a higher chance of survival.

The next strongest relationships were `Has_Cabin` (0.32) and `Fare` (0.26), both positively correlated with survival. This shows that survival was closely tied to wealth and access, not just chance.

The frequency counts also revealed important context about who was on board. Third class made up the largest group by far, with 491 out of 891 passengers (about 55%), more than first and second class combined. This means the disaster affected a large number of lower-class passengers, who also had the lowest survival rate — a combination that explains much of the overall loss of life. The dataset also showed nearly twice as many men as women aboard (577 vs. 314), and most passengers were young adults, with ages clustering between 20 and 40 years old.

Machine Learning Results

To take the analysis further, a Logistic Regression model was built to predict passenger survival using eight features: `Pclass`, `Sex`, `Age`, `SibSp`, `Parch`, `Fare`, `Embarked`, and `Has_Cabin`. The dataset was split so that 80% (712 passengers) was used to train the model, and the remaining 20% (179 passengers) was kept aside to test how well the model performed on data it had never seen.

The model achieved an accuracy of 81.01%, correctly predicting the outcome for 145 out of 179 test passengers. Looking more closely at the confusion matrix, the model correctly identified 97 out of 110 passengers who did not survive (a recall of 88%), but only correctly identified 48 out of 69 passengers who did survive (a recall of 70%). This means the model is noticeably better at recognizing non-survivors than survivors, likely because the training data itself contained more non-survivors (549) than survivors (342), giving the model more examples of that outcome to learn from.

The precision scores — 0.82 for non-survivors and 0.79 for survivors — show that when the model does make a prediction, it is correct roughly 4 out of 5 times regardless of which class it predicts. Overall, these machine learning results support and confirm the statistical findings: class, fare, and cabin status are strong, learnable predictors of survival, and even a relatively simple model can capture a large share of this pattern from the data.

Limitations of the Study

While the analysis produced clear and consistent findings, several limitations should be acknowledged:

- **Missing Age values** were filled using the median age of all passengers. While this is a reasonable estimate, it does not reflect each individual passenger's true age, and it created an artificial spike in the age distribution seen in the histogram.
- **The Cabin column** was missing for about 77% of passengers. Rather than losing this information entirely, it was converted into a simple yes/no flag (`Has_Cabin`). However, this approach loses more detailed information, such as which deck or section of the ship a passenger's cabin was located in, which could have been an even stronger predictor.
- **Logistic Regression** assumes a largely straight-line (linear) relationship between the features and the likelihood of survival. It may not fully capture more complex interactions, such as how age and sex combine to affect survival chances (for example, young children of either sex, or adult women specifically, may have had distinctly different survival patterns than the model can represent on its own).
- **The dataset only covers one historical event.** The Titanic disaster had unique circumstances, such as its location, the time it took to sink, and the number of lifeboats available. These findings should not be assumed to apply to other maritime disasters without further study.

Recommendations

Based on the findings and limitations above, the following are recommended for future work:

1. **Try more advanced models**, such as Random Forest or Gradient Boosting, and compare their accuracy against this Logistic Regression baseline to see if they capture the survival patterns more effectively.
2. **Engineer additional features**, such as extracting passenger titles (Mr., Mrs., Master, Miss) from the `Name` column, which often reflect age, gender, and social status more precisely, or combining `SibSp` and `Parch` into a single `FamilySize` feature to capture the effect of traveling with family.
3. **Use cross-validation** instead of a single train/test split, which would give a more reliable and stable estimate of how well the model performs, rather than relying on the results of just one random split.

Conclusion

This project successfully demonstrated a complete data science workflow: acquiring and cleaning real-world data, visualizing and statistically analyzing it, and building a machine learning model to make predictions. The analysis confirmed that survival on the Titanic was strongly shaped by passenger class, fare, and cabin access, reflecting the social and economic inequalities of the time. The Logistic Regression model, while simple, was able to predict survival with 81% accuracy, showing that these patterns are strong and learnable even with limited features. Overall, this project highlights both the value and the limitations of applying data science techniques to historical datasets.

That covers all six required points (Major findings, Statistical insights, ML results, Limitations, Recommendations) plus a closing conclusion — should comfortably fill close to two pages once pasted into Word with normal formatting. Want me to build the actual Word document next, with this content already dropped into the right sections alongside a cover page and table of contents?